



Système Intelligent Multi-Modèles pour la Maintenance Prédictive Industrielle

EFREI Paris — Mastère Data Engineering et Intelligence Artificielle — Promotion 2025-2026

Binôme	Khalil DJAHEL / Bryan BONTRAIN
Formatrice	Sarah MALAEB
Année scolaire	2025-2026
Date du rapport	Mai 2026

Projet certifiant RNCP36739 Expert en ingénierie de données

Bloc 4 : Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements et usages

Résumé Exécutif

La maintenance corrective industrielle coûte cher : arrêts non planifiés, réparations d'urgence, risques pour les opérateurs. Ce projet développe un système capable de détecter une panne 24 heures avant qu'elle survienne, à partir des données de capteurs déjà présents sur les machines.

Contexte d'application : machines industrielles de type CNC, pompes, compresseurs et bras robotiques, instrumentées avec des capteurs de vibration, température, pression et RPM.

Dataset : Kaggle, Industrial Machine Predictive Maintenance Dataset, 24 042 observations, 9 features retenues, variable cible binaire `failure_within_24h`.

Tâche prédictive : classification binaire supervisée. Le déséquilibre des classes (85 % / 15 %) est pris en charge nativement par chaque algorithme.

Modèles développés : quatre algorithmes ont été comparés, Logistic Regression (baseline), Random Forest (bagging), XGBoost (boosting) et un réseau de neurones MLP (Deep Learning).

Résultats : XGBoost est retenu avec $F1 = \mathbf{0.898}$, $\text{Recall} = \mathbf{0.955}$ et $\text{ROC-AUC} = \mathbf{0.996}$. La cross-validation 5-fold valide la stabilité du modèle ($F1 = 0.9026 \pm 0.0099$).

Outil final : dashboard Streamlit en 4 onglets (EDA, comparaison des modèles, prédiction en temps réel, interprétabilité SHAP), avec un pipeline sklearn sérialisé via joblib.

Technologies : Python, Pandas, scikit-learn, XGBoost, Keras/TensorFlow, SHAP, Streamlit, Matplotlib, Seaborn.

En synthèse : le projet part d'un dataset brut et aboutit à un outil opérationnel capable de détecter 19 pannes sur 20 avant qu'elles surviennent, avec une explication des alertes adaptée à un utilisateur non technique.

Table des matières

1. [Introduction et contexte](#)
 2. [Analyse du besoin utilisateur](#)
 3. [Méthodologie de travail et gestion de projet](#)
 4. [Référentiel de données](#)
 5. [Analyse Exploratoire des Données \(EDA\)](#)
 6. [Préparation et transformation des données](#)
 7. [Pipeline IA et architecture](#)
 8. [Implémentation technique — les modèles](#)
 9. [Évaluation comparative des modèles](#)
 10. [Analyse biais / variance et risques d'overfitting](#)
 11. [Interprétabilité du modèle final](#)
 12. [Interface utilisateur et dashboard décisionnel](#)
 13. [Résultats et tests de démonstration](#)
 14. [Gouvernance, responsabilité et limites](#)
 15. [Limites et pistes d'amélioration](#)
 16. [Conclusion et recommandations](#)
-

1. Introduction et contexte

1.1 Problématique industrielle

Les équipements industriels modernes (CNC, pompes, compresseurs, bras robotiques) génèrent en continu des données de capteurs physiques : vibrations, température moteur, pression, courant électrique, vitesse de rotation. Ces signaux portent des signatures annonçant les défaillances bien avant qu'elles se produisent, mais leur volume et leur vitesse rendent l'analyse manuelle impraticable.

Quand une machine tombe en panne sans prévenir, les conséquences sont immédiates : la ligne de production s'arrête, les équipes de maintenance interviennent dans l'urgence, les coûts s'envolent et les délais de livraison sont compromis. Dans certains secteurs, une heure d'arrêt non planifié représente plusieurs dizaines de milliers d'euros de perte.

La maintenance prédictive renverse cette logique : au lieu d'attendre la panne pour réagir, on anticipe en exploitant les données capteurs pour planifier les interventions au bon moment.

1.2 Objectifs du projet

Le projet vise à construire un MVP de plateforme de maintenance prédictive comprenant quatre composantes principales :

1. Un pipeline de préparation des données robuste (nettoyage, encodage, normalisation, anti-leakage)
2. La comparaison rigoureuse de quatre modèles ML/DL sur un jeu de test commun
3. Un dashboard Streamlit destiné à un responsable maintenance non technique
4. Une analyse d'interprétabilité via Feature Importance et SHAP

1.3 Tâche prédictive retenue

La variable cible choisie est `failure_within_24h` (0 = pas de panne imminente, 1 = panne attendue dans les 24 heures). C'est la tâche la plus directement exploitable opérationnellement : elle laisse suffisamment de temps pour planifier une intervention préventive.

Le dataset contient d'autres cibles possibles (`failure_type` , `rul_hours`) mais elles ont été écartées car elles constituent un data leakage, comme expliqué en section 4.

2. Analyse du besoin utilisateur

2.1 Utilisateur cible

L'utilisateur principal est un responsable maintenance ou un ingénieur opérationnel chargé de surveiller l'état des équipements. Ce profil connaît ses machines et leurs comportements habituels, mais n'est pas data scientist. Il a besoin d'une réponse claire et rapide, pas d'un score de probabilité brut.

Trois profils ont été identifiés comme utilisateurs potentiels :

- Le responsable maintenance qui planifie les interventions préventives sur la semaine
- L'ingénieur de production qui surveille les performances des équipements en temps réel
- Le manager opérationnel qui cherche à réduire le taux d'arrêts non planifiés sur son site

2.2 Scénarios d'usage

Scénario	Besoin	Réponse de la solution
Surveillance quotidienne	Quelles machines nécessitent une attention particulière ?	Score de risque par machine dans l'onglet EDA
Alerte imminente	Cette machine doit-elle être arrêtée avant panne ?	Probabilité de panne et recommandation d'intervention
Bilan de confiance	Le système est-il fiable sur la durée ?	Tableau comparatif des modèles, courbes ROC
Compréhension d'une alerte	Pourquoi cette machine est-elle classée à risque ?	Analyse SHAP des capteurs déclencheurs

2.3 Contraintes identifiées

- L'utilisateur doit comprendre le résultat en moins de cinq secondes, sans formation technique préalable
- La prédiction doit être instantanée après saisie des valeurs capteurs
- Chaque alerte doit s'accompagner d'une explication concrète (quel capteur, quel seuil)
- La solution doit tolérer les fausses alertes plutôt que de rater une vraie panne

2.4 Indicateurs attendus dans le dashboard

- Score de risque coloré (ÉLEVÉ / MODÉRÉ / FAIBLE)

- Probabilité de panne en pourcentage
- Message de recommandation en langage naturel
- Classement des variables ayant le plus contribué à la prédiction

2.5 Valeur ajoutée

Avec un Recall de 95.5 %, le système détecte 19 pannes sur 20 avant qu'elles surviennent. Dans un contexte industriel où chaque heure d'arrêt non planifié peut coûter plusieurs milliers d'euros, cette capacité d'anticipation a une valeur économique directe et mesurable.

3. Méthodologie de travail et gestion de projet

3.1 Approche adoptée

Le projet a été organisé en itérations courtes selon une logique Agile/Kanban. Cette méthode s'est avérée utile en pratique : quand les premiers résultats de la régression logistique ont montré ses limites sur un dataset déséquilibré, il a été possible de réorienter rapidement les efforts vers les modèles ensemblistes.

3.2 Répartition des tâches

Tâche	Responsable
Analyse exploratoire (EDA) et visualisations	Bryan BONTRAIN
Pipeline preprocessing et anti-leakage	Khalil DJAHEL
Modèles Machine Learning (LR, RF, XGBoost)	Khalil DJAHEL
Modèle Deep Learning (MLP Keras)	Khalil DJAHEL
Analyse SHAP et interprétabilité	Khalil DJAHEL
Dashboard Streamlit	Bryan BONTRAIN
Rédaction du rapport	Binôme
Préparation soutenance	Binôme

Outils utilisés : GitHub pour le versionnage du code, Jupyter Notebooks pour l'exploration, Google Drive pour les livrables partagés.

3.3 Risques rencontrés et solutions

Risque	Impact observé	Solution appliquée
Déséquilibre des classes (85/15)	Sans correction, les modèles prédisaient systématiquement "pas de panne"	<code>class_weight='balanced'</code> pour LR et RF, <code>scale_pos_weight=5.75</code> pour XGBoost
Data leakage	Scores artificiellement élevés sur le test set	Suppression de <code>failure_type</code> , <code>rul_hours</code> , <code>estimated_repair_cost</code> et encapsulation dans des sklearn Pipelines
Overfitting du MLP	Le réseau mémorisait le train set sans généraliser	<code>early_stopping=True</code> et régularisation <code>alpha=1e-4</code>
Performance limitée du Deep Learning	MLP inférieur à XGBoost sur données tabulaires	Analyse critique intégrée au rapport, sans forcer l'utilisation du DL

4. Référentiel de données

Source : Kaggle, Industrial Machine Predictive Maintenance Dataset.

Fichier : `predictive_maintenance_v3.csv` .

4.1 Caractéristiques générales

Attribut	Valeur
Nombre d'enregistrements	24 042
Nombre de variables	15
Période couverte	2024 (données simulées)
Fréquence d'échantillonnage	Environ toutes les 3 minutes
Types de machines	CNC, Pump, Compressor, Robotic Arm

4.2 Variables du dataset

Variable	Type	Description	Statut
timestamp	datetime	Horodatage de la mesure	Supprimé (identifiant)
machine_id	int	Identifiant de la machine	Supprimé (identifiant)
machine_type	string	Type de machine	Feature catégorielle
vibration_rms	float	Vibration RMS	Feature numérique
temperature_motor	float	Température moteur (°C)	Feature numérique
current_phase_avg	float	Courant électrique moyen (A)	Feature numérique
pressure_level	float	Niveau de pression	Feature numérique
rpm	float	Vitesse de rotation	Feature numérique
operating_mode	string	Mode opératoire (idle/normal/peak)	Feature catégorielle
hours_since_maintenance	float	Heures depuis dernière maintenance	Feature numérique
ambient_temp	float	Température ambiante (°C)	Feature numérique
rul_hours	float	Durée de vie restante estimée	Supprimé (data leakage)
failure_within_24h	int	Variable cible (0/1)	Cible
failure_type	string	Type de panne	Supprimé (data leakage)
estimated_repair_cost	int	Coût estimé de réparation	Supprimé (data leakage)

4.3 Justification de la suppression des variables à risque de leakage

Le data leakage désigne l'utilisation pendant l'entraînement d'informations qui ne seraient pas disponibles au moment d'une prédiction réelle. C'est l'une des erreurs les plus courantes en Data Science : le modèle affiche des scores excellents en test, puis échoue en production parce qu'il a appris à tricher.

Trois variables ont été supprimées pour cette raison :

- `failure_type` révèle directement le type de panne en cours. Si le modèle peut lire cette colonne, il sait déjà qu'il y a une panne, ce qui rend la prédiction triviale et sans intérêt.
- `rul_hours` (durée de vie restante estimée) encode implicitement la variable cible. Quand `rul_hours` est inférieur à 24, `failure_within_24h` vaut presque toujours 1. La corrélation mesurée est de -0.25, ce qui confirme cette dépendance.
- `estimated_repair_cost` est calculé après la survenue d'une panne et ne serait donc jamais disponible au moment de la prédiction.

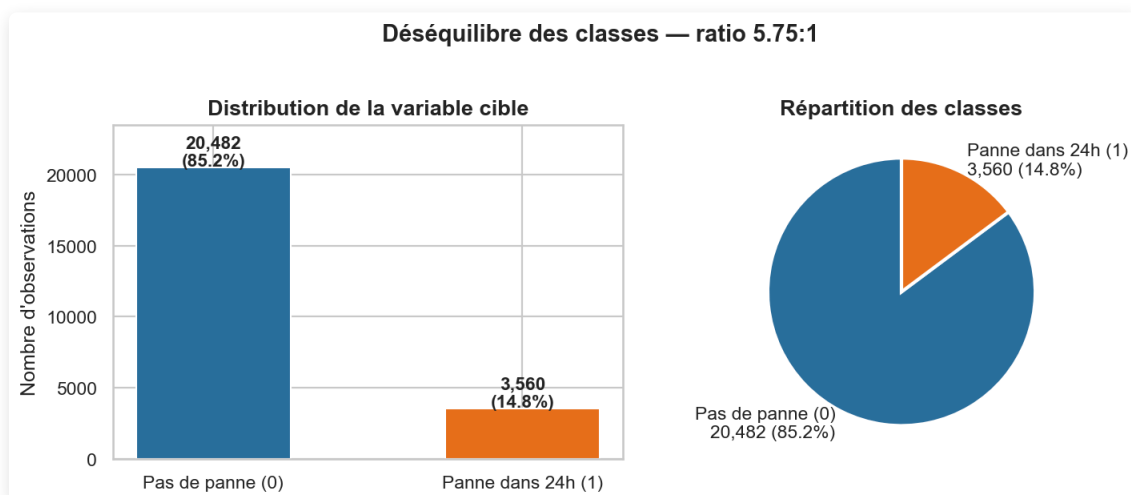
4.4 Limites du dataset

Le dataset est généré synthétiquement. Les performances mesurées pourraient être inférieures sur des données industrielles réelles, où le bruit de capteur, les pannes atypiques et la variabilité entre machines compliquent la tâche. Par ailleurs, chaque enregistrement est traité comme un instantané indépendant : les tendances d'évolution progressive des capteurs (un roulement qui se dégrade sur plusieurs heures) ne sont pas exploitées. Enfin, des informations contextuelles utiles comme l'âge des équipements ou leur historique complet de pannes ne sont pas disponibles dans ce dataset.

5. Analyse Exploratoire des Données (EDA)

5.1 Distribution de la variable cible

Classe	Nombre	Pourcentage
0 — Pas de panne	20 482	85.2 %
1 — Panne dans 24h	3 560	14.8 %



Le déséquilibre (85 % / 15 %) est réaliste dans notre situation mais sans traitement adapté, les modèles tendraient à prédire systématiquement "pas de panne" et afficheraient 85 % d'accuracy sans jamais détecter une seule panne réelle, ce qui est parfaitement inutile.

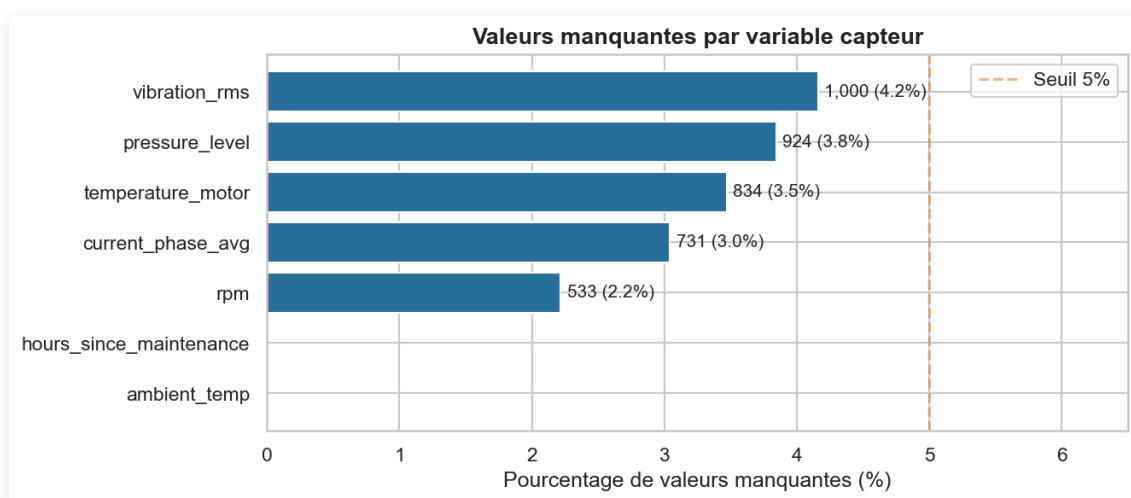
5.2 Répartition par type de machine

Machine	Observations
CNC	25 %
Pump	25 %
Compressor	25 %
Robotic Arm	25 %

La répartition équilibrée entre les quatre types de machines garantit que les modèles ne seront pas biaisés vers un type particulier d'équipement.

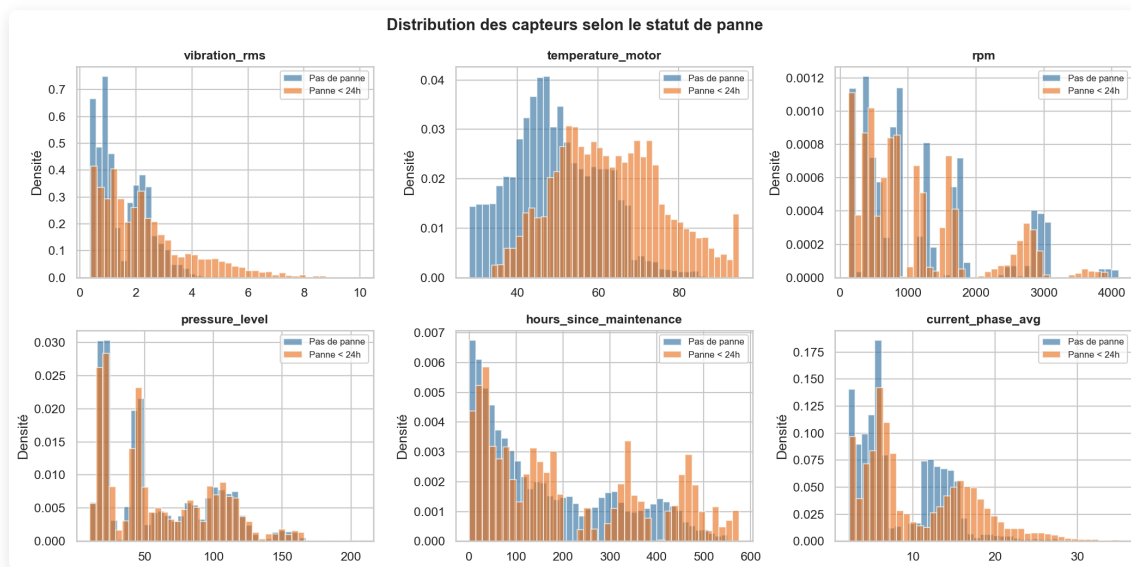
5.3 Valeurs manquantes

Variable	Valeurs manquantes	%
vibration_rms	1 000	4.2 %
temperature_motor	834	3.5 %
current_phase_avg	731	3.0 %
pressure_level	924	3.8 %
rpm	533	2.2 %



Les valeurs manquantes, comprises entre 2 % et 4 % selon le capteur, proviennent probablement de défaillances temporaires de transmission ou d'interruptions de communication réseau. L'imputation par la médiane a été choisie car elle est insensible aux valeurs extrêmes, contrairement à la moyenne qui serait faussée par les pics de vibration ou les épisodes de surchauffe.

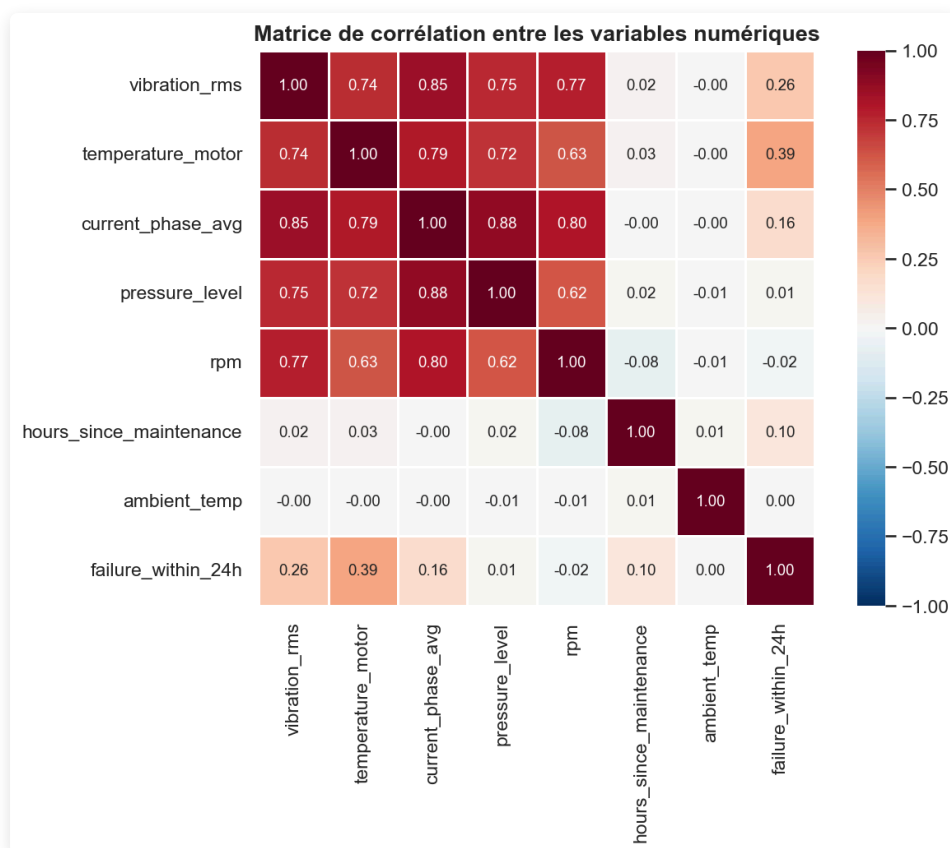
5.4 Distributions des capteurs par statut de panne



L'analyse des distributions fait apparaître des patterns nets et physiquement cohérents :

- **vibration_rms** : les observations de classe 1 présentent des valeurs systématiquement plus élevées. Une vibration anormale est un précurseur classique de dégradation mécanique (roulements usés, balourd, desserrage).
- **temperature_motor** : une surchauffe du moteur est fortement associée aux pannes imminentes, ce qui correspond au frottement accru lors de la dégradation.
- **hours_since_maintenance** : les machines dont la dernière maintenance remonte à plus longtemps sont davantage représentées dans la classe 1, ce qui reflète l'usure progressive.
- **operating_mode** : le mode **peak** est sur-représenté parmi les observations de panne, cohérent avec le stress mécanique supérieur qu'il génère.

5.5 Matrice de corrélations



La matrice de corrélation ne révèle pas de redondance forte entre les features. Les corrélations les plus notables avec la variable cible concernent `vibration_rms` et `temperature_motor`, ce qui confirme les observations des distributions.

6. Préparation et transformation des données

6.1 Étapes du pipeline de préparation

```
Données brutes (24 042 lignes, 15 colonnes)
|
Suppression des colonnes (identifiants + data leakage)
|
Séparation features / cible
|
Split stratifié 80/20 (train: 19 233 | test: 4 809)
|
ColumnTransformer (ajusté sur TRAIN uniquement, appliqué sur TEST)
|-- Numériques (7 variables) : SimpleImputer(median) puis StandardScaler
|-- Catégorielles (2 variables) : SimpleImputer(mode) puis OneHotEncoder
```

Le split stratifié signifie que la proportion de pannes (14.8 %) est préservée à l'identique dans le train set et dans le test set. Sans stratification, les pannes pourraient se retrouver inégalement réparties entre les deux ensembles, faussant l'évaluation.

6.2 Prévention du data leakage avec sklearn Pipelines

L'ensemble du preprocessing est encapsulé dans des sklearn Pipelines. Toutes les statistiques de preprocessing (médiane pour l'imputation, paramètres du StandardScaler, catégories pour l'OneHotEncoder) sont calculées uniquement sur les données d'entraînement, puis appliquées sur le test set. Le modèle n'a jamais accès aux données de test pendant la phase d'apprentissage.

6.3 Stratégie de gestion du déséquilibre de classes

Modèle	Stratégie	Explication
Logistic Regression	<code>class_weight='balanced'</code>	sklearn recalcule des poids inversement proportionnels à la fréquence de chaque classe
Random Forest	<code>class_weight='balanced'</code>	Chaque panne compte 5.75 fois plus qu'une non-panne dans la fonction de coût
XGBoost	<code>scale_pos_weight=5.75</code>	Paramètre natif : ratio 20 482 / 3 560
MLP	<code>early_stopping=True</code>	Arrête l'entraînement quand les performances sur l'ensemble de validation se dégradent

6.4 Métriques d'évaluation

Les quatre cas de figure possibles pour chaque prédiction :

	Prédit : PAS de panne	Prédit : PANNE
Réel : PAS de panne	TN (Vrai Négatif)	FP (Faux Positif, fausse alerte)
Réel : PANNE	FN (Faux Négatif, panne ratée)	TP (Vrai Positif)

Dans un contexte industriel, le faux négatif est le cas le plus coûteux : le modèle dit que tout va bien alors qu'une panne arrive. C'est pour cette raison que le Recall a été choisi comme métrique principale.

Recall = $TP / (TP + FN)$ — parmi toutes les vraies pannes, quelle proportion le modèle détecte-t-il ?

Precision = $TP / (TP + FP)$ — parmi toutes les alertes, quelle proportion est justifiée ?

F1-Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — moyenne harmonique, utilisée comme critère de sélection du modèle final.

ROC-AUC — aire sous la courbe ROC, qui trace le Recall en fonction du taux de faux positifs pour tous les seuils de décision possibles. Une valeur de 1 correspond à un modèle parfait, 0.5 à une prédiction aléatoire.

6.5 Validation croisée

Une cross-validation stratifiée 5-fold a été appliquée sur le modèle retenu (XGBoost). Le dataset est découpé en 5 partitions : le modèle est entraîné 5 fois, chaque fois évalué sur une partition différente. Un écart-type faible entre les 5 scores confirme que les performances ne dépendent pas d'un découpage particulier et que le modèle généralise bien.

7. Pipeline IA et architecture

Le schéma ci-dessous représente la chaîne de traitement complète, du chargement des données brutes jusqu'au dashboard décisionnel.

```

+-----+
|                                     |
|                               DONNEES BRUTES                               |
|                                     |
+-----+
|                                     |
|                               +-----+                               |
|                               | Suppression |                               |
|                               | leakage +   |                               |
|                               | identifiants |                               |
|                               +-----+                               |
|                                     |
+-----+
| Split stratifie 80/20 |
| Train : 19 233 obs.  |
| Test  : 4 809 obs.   |
+-----+
|                               |
|                               +-----+                               |
| ColumnTransf. |
| (sur TRAIN) |
| Imputer      |
| Scaler       |
| Encoder      |
+-----+
| (applique aussi sur TEST) |
|                               |
+-----+
| ENTRAINEMENT (4 modeles) |
| Logistic Regression | Random Forest |
| XGBoost            | MLP Deep Learning |
+-----+
|                               |
|                               +-----+                               |
| EVALUATION sur TEST SET   +-----+                               |
| Recall / F1 / ROC-AUC    |                               |
| Matrices de confusion    |                               |
| Cross-validation 5-fold   |                               |
+-----+
|                               |
+-----+
| MODELE RETENU : XGBoost |
| Serialisation joblib   |
+-----+
|                               |
+-----+
| INTERPRETABILITE SHAP |
| Feature Importance | TreeExplainer |
+-----+
|                               |
+-----+
| DASHBOARD STREAMLIT |
+-----+

```

Technologies par composant :

Composant	Technologies
Chargement et exploration	Pandas, Matplotlib, Seaborn
Preprocessing	scikit-learn (ColumnTransformer, Pipelines)
Modèles ML	scikit-learn (LR, RF), XGBoost
Modèle DL	Keras / TensorFlow (MLP)
Interprétabilité	SHAP (TreeExplainer)
Sérialisation	joblib
Dashboard	Streamlit

8. Implémentation technique — les modèles

8.1 Logistic Regression — modèle baseline

Principe : le modèle calcule une combinaison linéaire pondérée des features ($z = w_1 \times \text{vibration} + w_2 \times \text{température} + \dots$) puis applique la fonction sigmoïde $\sigma(z) = 1/(1+e^{(-z)})$ pour obtenir une probabilité entre 0 et 1. Les poids w sont appris par minimisation de la log-vraisemblance.

Paramètres : `C=1.0` , `solver='lbfgs'` , `max_iter=1000` , `class_weight='balanced'`

La régression logistique est très interprétable (un poids positif signifie que la feature augmente le risque de panne) et sert de référence : si ce modèle surpassait les autres, le problème serait linéairement séparable et la complexité supplémentaire des autres modèles serait inutile.

Sa limite principale est précisément cette linéarité : il ne peut pas capturer des interactions non linéaires du type "si la vibration ET la température sont simultanément élevées depuis plusieurs heures".

8.2 Random Forest — modèle ensembliste (Bagging)

Principe : 200 arbres de décision sont entraînés en parallèle, chacun sur un sous-échantillon aléatoire des données. Chaque arbre pose des questions binaires successives (`vibration_rms > 2.3 ?`) jusqu'à une décision finale. La prédiction du modèle est la moyenne des 200 probabilités individuelles.

La double randomisation (sous-ensemble d'observations ET sous-ensemble de features à chaque nœud) force les arbres à faire des erreurs différentes. En moyennant ces erreurs non corrélées, elles se compensent — c'est le principe du bagging (Bootstrap AGGREGatING).

Paramètres : `n_estimators=200` , `max_depth=15` , `class_weight='balanced'`

Le Random Forest capture les non-linéarités, est robuste aux outliers et fournit une feature importance native. Son inconvénient sur ce projet est son volume (21 MB sérialisé) comparé aux 600 KB de XGBoost, ce qui le défavorise pour un déploiement en production.

8.3 XGBoost — Gradient Boosting

Principe : contrairement au Random Forest, les 200 arbres sont construits séquentiellement. Chaque arbre est entraîné pour corriger les erreurs du précédent. La direction de correction est guidée par le gradient de la fonction de perte, ce qui concentre l'effort d'apprentissage là où les erreurs sont les plus importantes.

Avec `learning_rate=0.1` , chaque arbre ne corrige que 10 % des erreurs résiduelles de manière contrôlée. La régularisation L1/L2 intégrée limite l'overfitting sans paramétrage supplémentaire.

Paramètres : `n_estimators=200` , `max_depth=6` , `learning_rate=0.1` , `scale_pos_weight=5.75`

XGBoost est reconnu dans la littérature comme le meilleur modèle sur les données tabulaires structurées, ce que nos résultats confirment.

8.4 MLP — Deep Learning (Réseau de neurones multicouche)

Principe : un réseau de neurones à 3 couches cachées. Chaque neurone calcule $z = \sum(w_i \times x_i) + b$ puis applique l'activation ReLU : $\max(0, z)$. L'entraînement se fait par rétropropagation du gradient : l'erreur en sortie remonte couche par couche pour ajuster tous les poids du réseau.

Architecture : 11 features en entrée, 128 neurones en couche 1, 64 en couche 2, 32 en couche 3, 1 neurone de sortie avec activation sigmoïde (probabilité de panne).

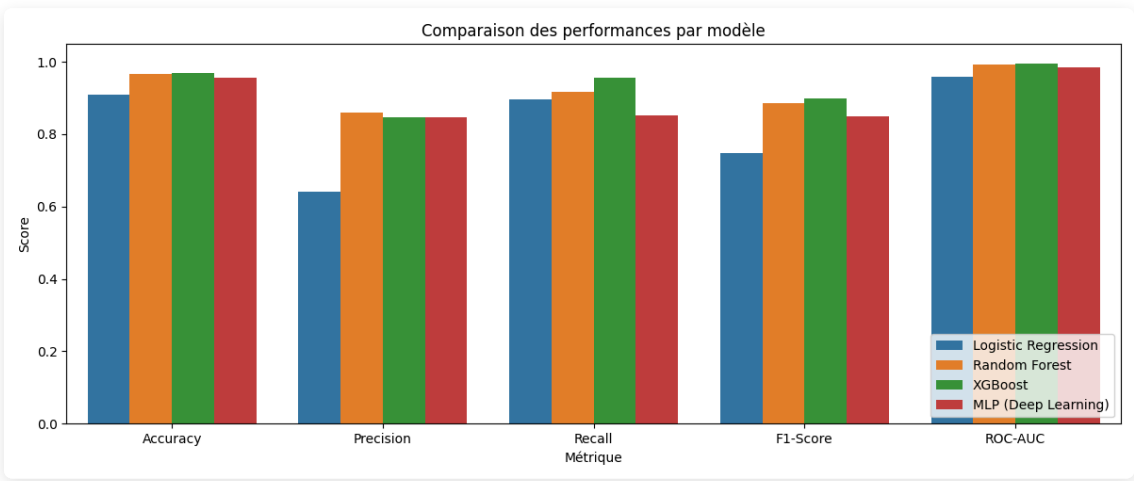
Paramètres : `hidden_layer_sizes=(128, 64, 32)` , `activation='relu'` , `alpha=1e-4` , `early_stopping=True`

Le MLP peut capturer des interactions complexes entre features sans feature engineering manuel. Sur ce projet, il est moins performant que XGBoost (F1 = 0.850 vs 0.898) car les données tabulaires structurées avec seulement 24 000 observations ne permettent pas au réseau d'exploiter pleinement sa capacité. Ce résultat est analysé en détail en section 15.

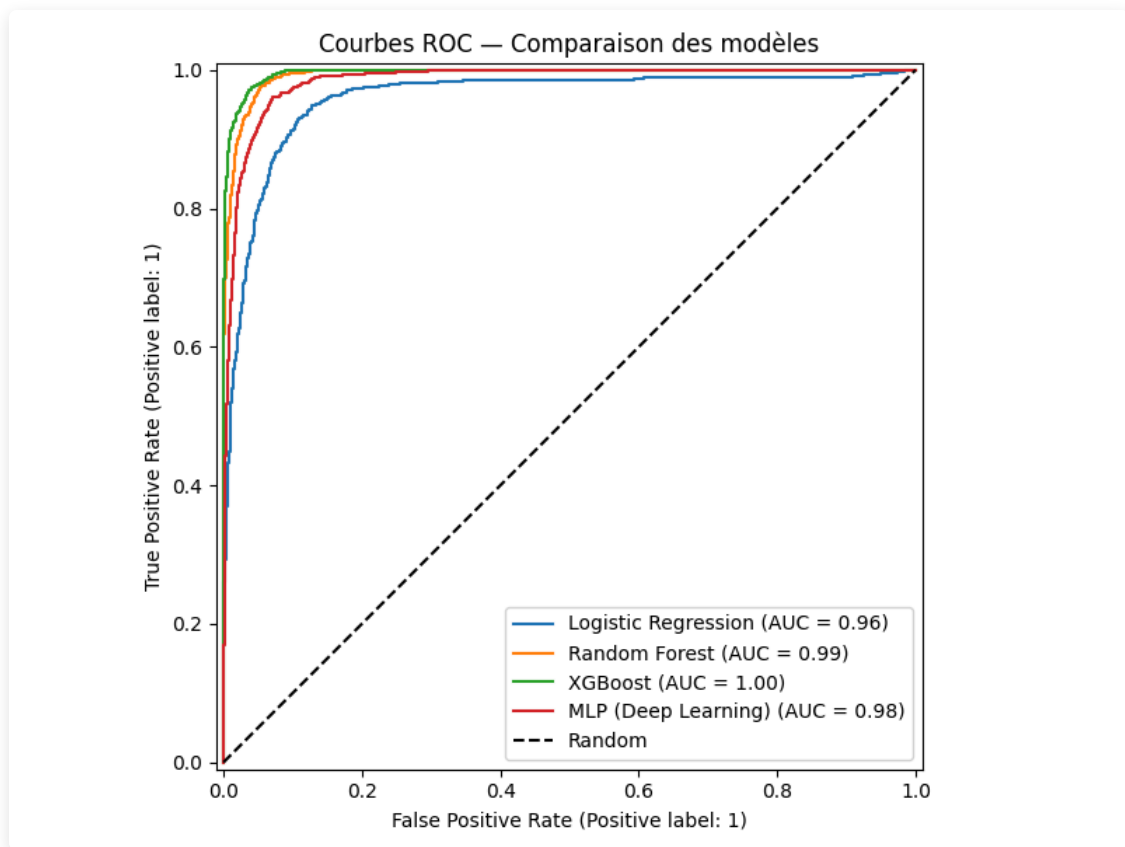
9. Évaluation comparative des modèles

9.1 Tableau des performances (ensemble de test, n = 4 809)

Modèle	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.910	0.641	0.895	0.747	0.959
Random Forest	0.965	0.859	0.916	0.887	0.993
XGBoost ✓	0.968	0.847	0.955	0.898	0.996
MLP (Deep Learning)	0.955	0.847	0.853	0.850	0.984



9.2 Courbes ROC des 4 modèles



9.3 Analyse des résultats

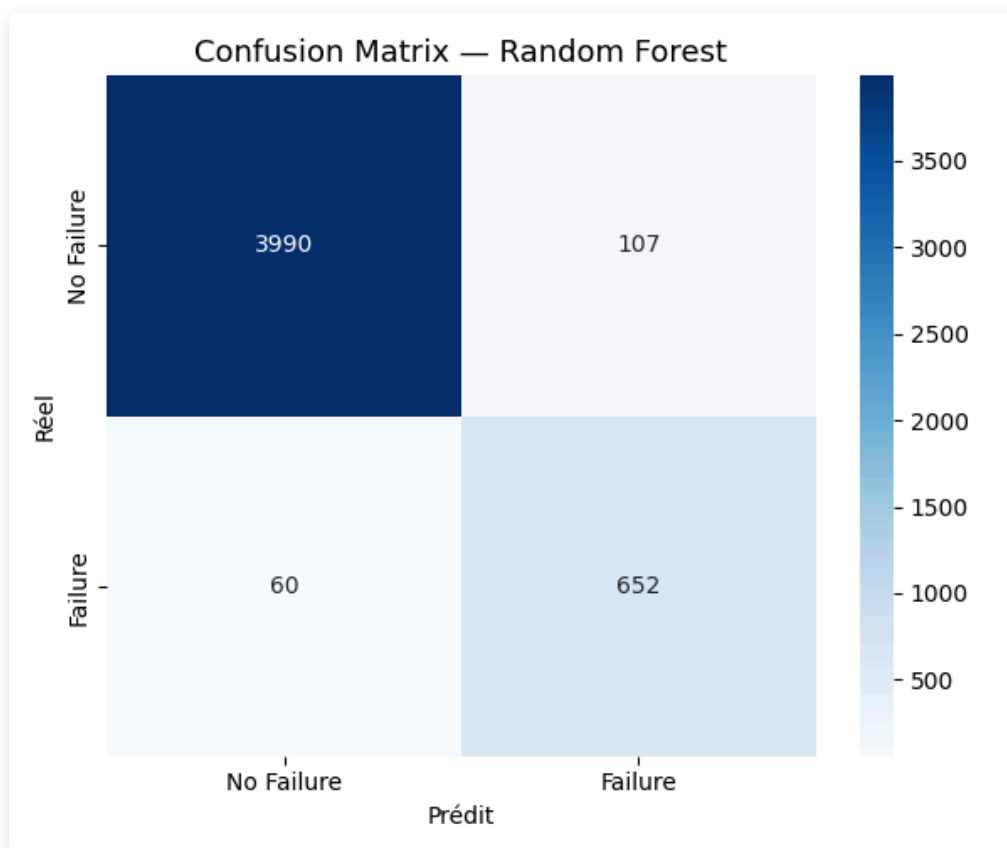
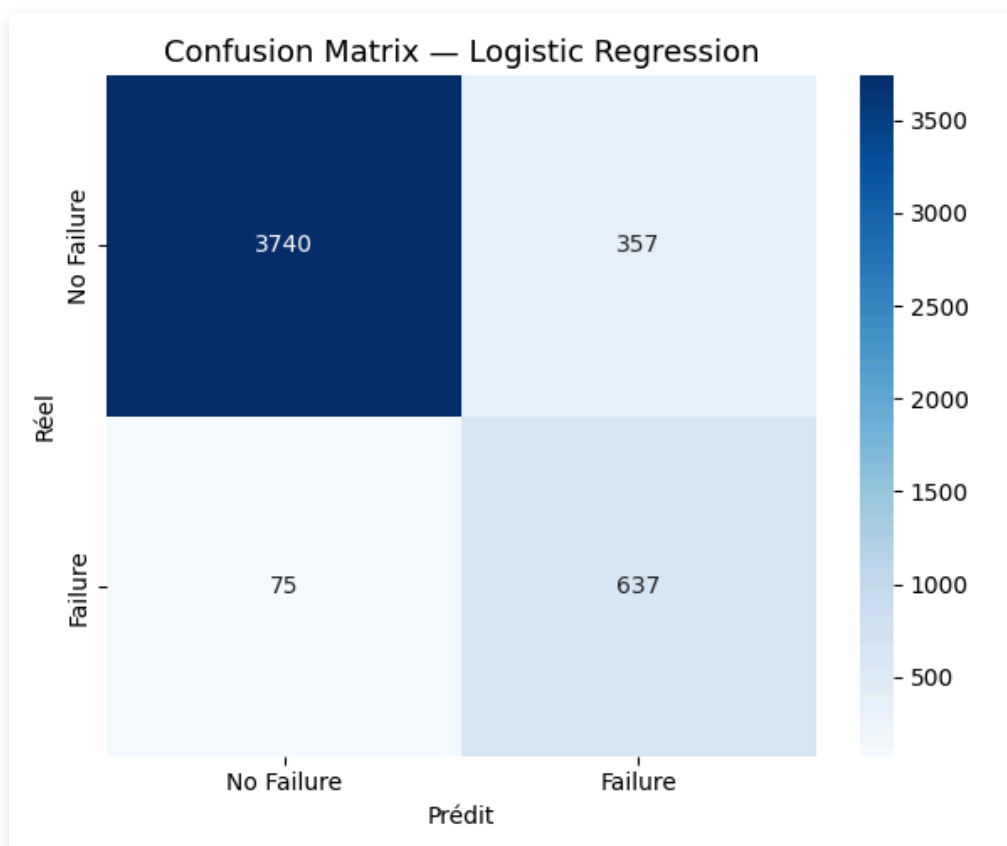
Logistic Regression : la Precision faible (0.641) génère de nombreuses fausses alertes. Le modèle linéaire ne parvient pas à capturer les interactions entre capteurs. Son rôle de baseline est confirmé par le fait que tous les autres modèles le surpassent avec une marge significative.

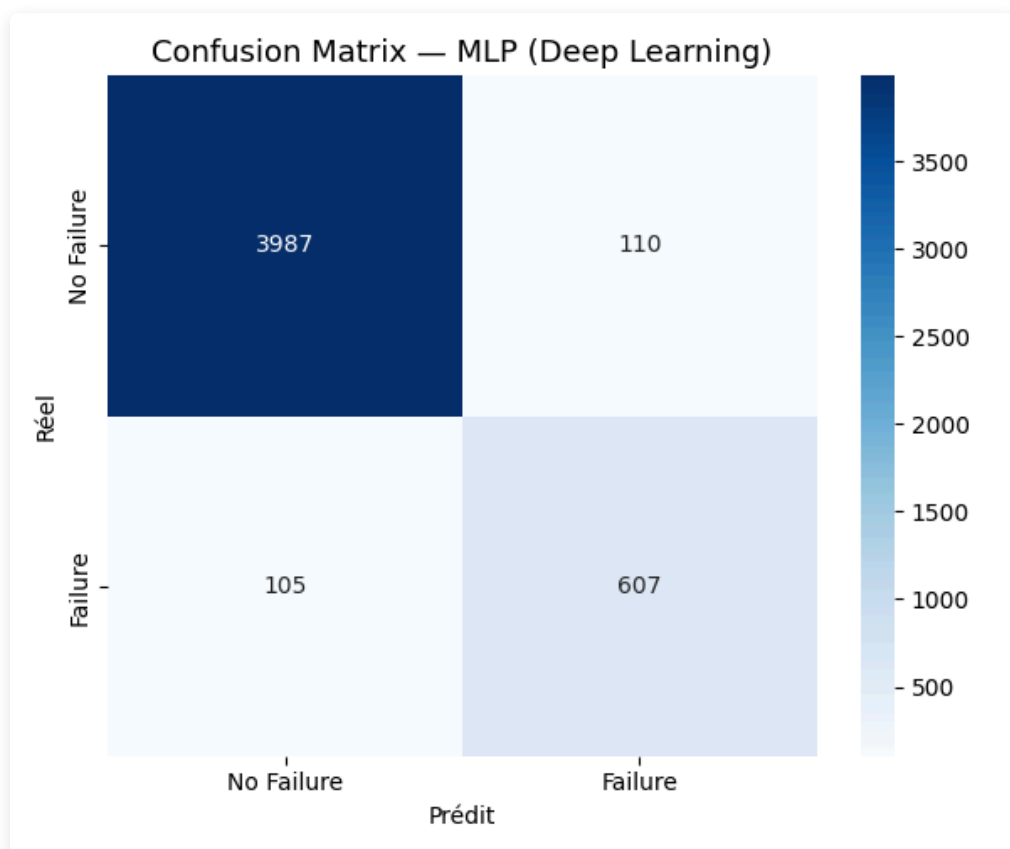
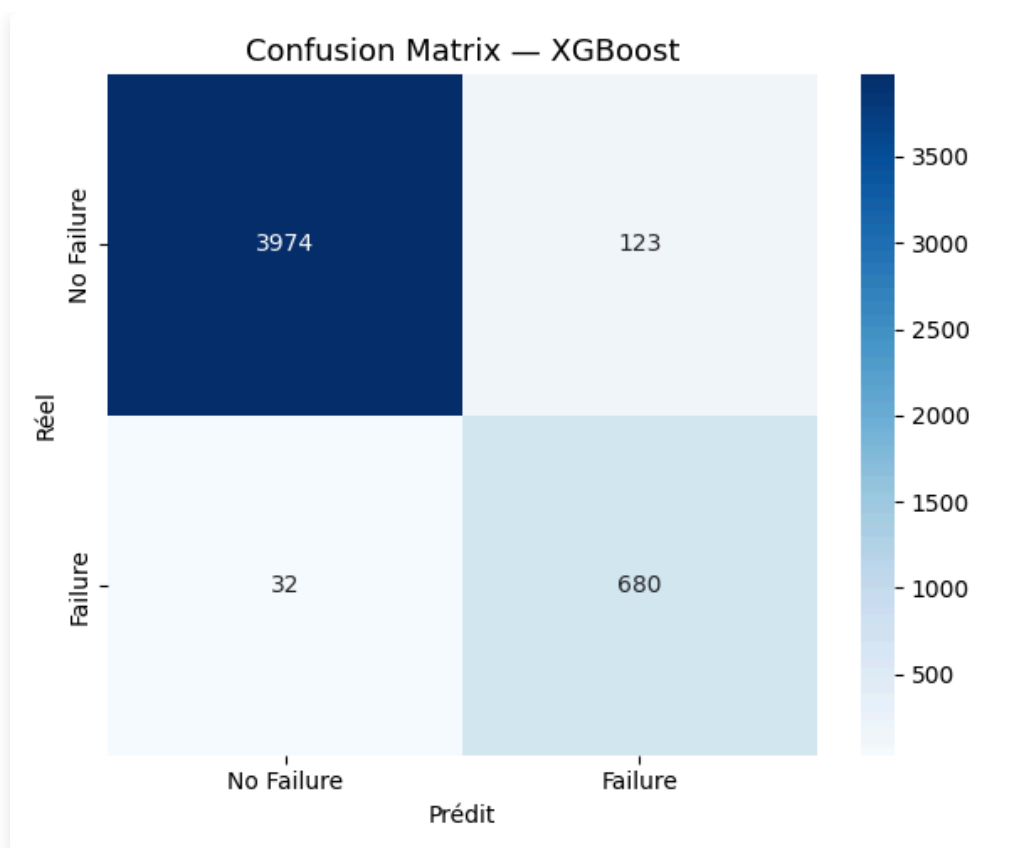
Random Forest : les performances sont très solides (F1 = 0.887, ROC-AUC = 0.993). Le gain de 0.14 point de F1 par rapport à la régression logistique confirme que les données contiennent des relations non linéaires importantes. XGBoost le dépasse néanmoins sur toutes les métriques, et son volume sérialisé (21 MB) complique le déploiement.

XGBoost (modèle retenu) : meilleur compromis sur l'ensemble des métriques. Un Recall de **0.955** signifie que 95.5 % des pannes réelles sont détectées, soit 19 sur 20. Le ROC-AUC de 0.996 indique une quasi-parfaite séparation entre les deux classes.

MLP (Deep Learning) : F1 = 0.850, inférieur à XGBoost. Avec 24 000 observations et des features tabulaires bien définies, les arbres de décision ont l'avantage structurel sur les réseaux de neurones. Ce résultat est cohérent avec la littérature récente sur les données tabulaires.

9.4 Matrices de confusion





9.5 Cross-validation XGBoost (5-fold stratifié)

Fold	F1-Score
1	0.8930
2	0.8915
3	0.9105
4	0.9008
5	0.9169
Moyenne	0.9026
Écart-type	0.0099

Un écart-type de 0.01 entre les cinq folds confirme que le modèle est stable : les performances sont reproductibles quelle que soit la partition utilisée.

9.6 Justification du choix du modèle final

Critère	Évaluation
Performance (F1, ROC-AUC)	Meilleur de tous les modèles testés
Recall	0.955, prioritaire en contexte industriel
Stabilité (CV 5-fold)	Faible variance entre folds (0.0099)
Gestion du déséquilibre	Paramètre <code>scale_pos_weight</code> natif
Interprétabilité	Feature importance et SHAP via TreeExplainer
Coût computationnel	Entraînement en environ 30 secondes
Déploiement	Modèle sérialisé léger (600 KB contre 21 MB pour RF)

10. Analyse biais / variance et risques d'overfitting

10.1 Le compromis biais / variance

Tout modèle de Machine Learning navigue entre deux écueils opposés :

- Biais élevé (underfitting) : le modèle est trop simple pour capturer les patterns des données. Les erreurs sont élevées aussi bien sur le train que sur le test.
- Variance élevée (overfitting) : le modèle a mémorisé les données d'entraînement, y compris leur bruit. Les performances sur le train sont excellentes, mais chutent sur le test.

L'objectif est de maintenir des performances proches et élevées sur les deux ensembles.

10.2 Diagnostic par modèle

Modèle	Biais	Variance	Diagnostic
Logistic Regression	Élevé	Faible	Underfitting : modèle trop simple pour les non-linéarités du dataset
Random Forest	Faible	Faible	Bon équilibre, le bagging réduit la variance naturellement
XGBoost	Faible	Très faible	Excellent équilibre, la régularisation L1/L2 intégrée aide
MLP (Deep Learning)	Modéré	Modéré	Risque d'overfitting sur la classe majoritaire, corrigé par <code>early_stopping</code>

10.3 Mesures appliquées pour contrôler l'overfitting

Pour le Random Forest : `max_depth=15` limite la profondeur des arbres et `class_weight='balanced'` équilibre l'apprentissage entre les deux classes.

Pour XGBoost : `max_depth=6` maintient des arbres peu profonds, `learning_rate=0.1` ralentit la correction pour éviter le surajustement, et la régularisation L1/L2 pénalise les poids trop élevés.

Pour le MLP : `early_stopping=True` interrompt l'entraînement dès que la validation loss cesse de diminuer, et `alpha=1e-4` applique une régularisation L2 sur les poids du réseau.

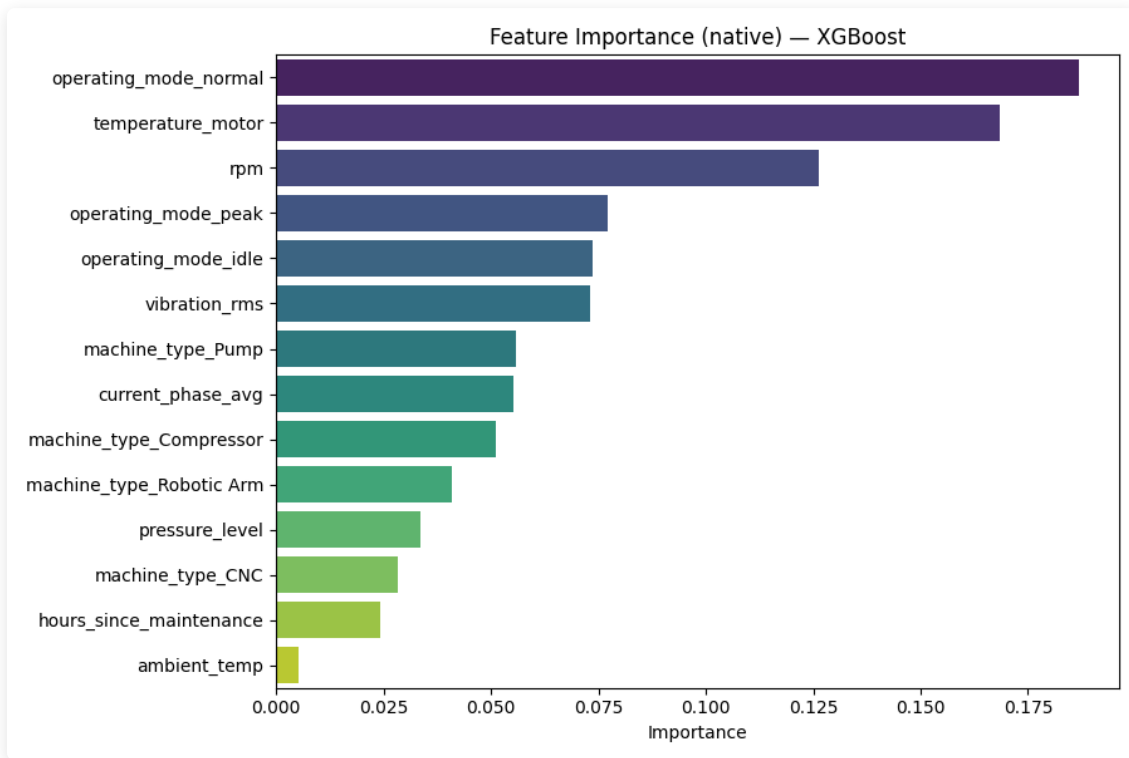
10.4 Preuve empirique de généralisation

La cross-validation 5-fold sur XGBoost donne $F1 = 0.9026 \pm 0.0099$ sur des données jamais vues pendant l'entraînement. Le score sur le test set isolé est de 0.898, soit un écart de moins de 0.005. Un modèle en overfitting afficherait un écart bien plus important. La faible variance entre les folds (0.0099) confirme que les performances sont reproductibles et que le modèle est fiable.

11. Interprétabilité du modèle final

11.1 Feature Importance native (XGBoost)

XGBoost calcule l'importance de chaque variable par le gain moyen apporté lors des splits dans l'ensemble des arbres. Plus une variable est utilisée fréquemment et plus les splits qu'elle génère réduisent l'erreur, plus son score d'importance est élevé.



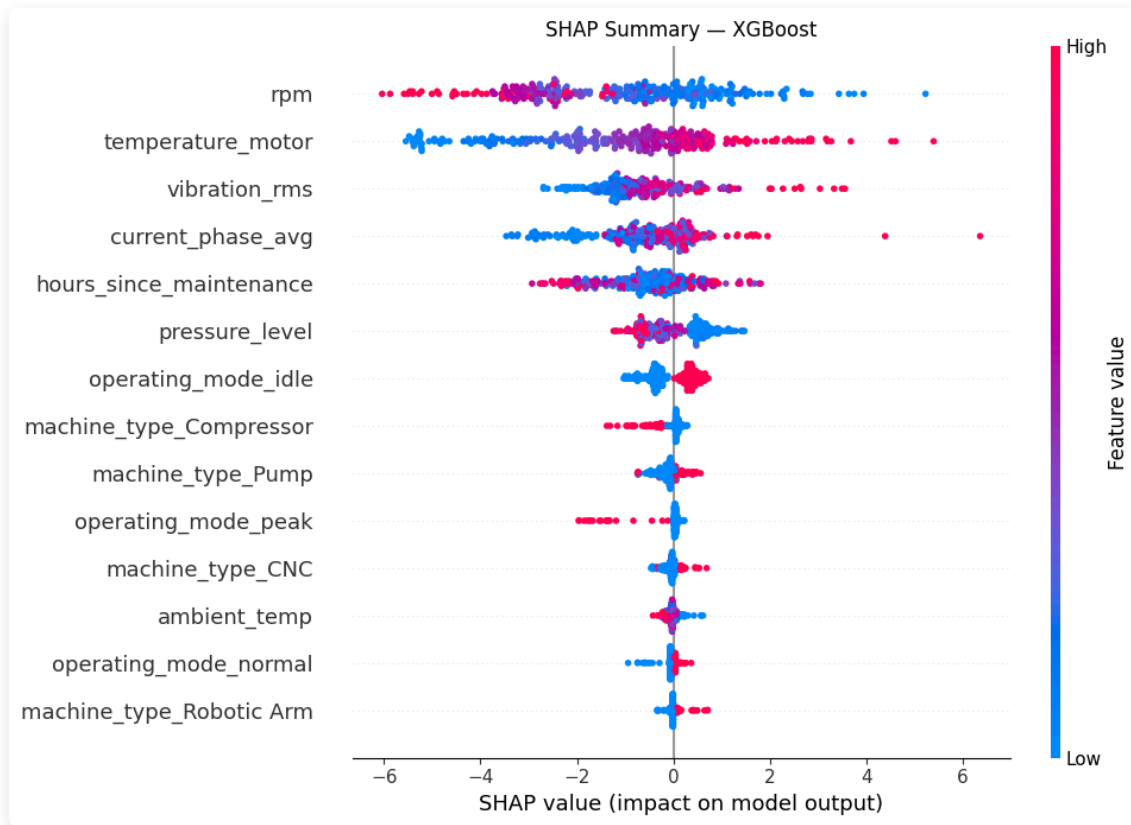
Top 5 variables par importance :

1. `vibration_rms` : indicateur principal de dégradation mécanique (roulements usés, balourd, desserrage)
2. `temperature_motor` : précurseur de défaillances thermiques (isolation endommagée, surcharge prolongée)
3. `hours_since_maintenance` : mesure de l'usure accumulée depuis la dernière intervention
4. `rpm` : stress mécanique lié à la vitesse de rotation
5. `pressure_level` : anomalie hydraulique ou pneumatique

Ces résultats sont physiquement cohérents avec ce qu'un technicien de maintenance expérimenté surveille en priorité, ce qui valide que le modèle a appris des patterns réels plutôt que du bruit statistique.

11.2 SHAP (SHapley Additive exPlanations)

La Feature Importance donne une vision globale du comportement du modèle sur l'ensemble du dataset. SHAP complète cette analyse en décomposant chaque prédiction individuelle en contributions de chaque feature, sur la base de la théorie des jeux coopératifs.



Lecture du graphique :

- Axe X positif : la feature contribue à prédire une panne
- Axe X négatif : la feature contribue à prédire l'absence de panne
- Rouge = valeur élevée de la feature, Bleu = valeur faible
- Ordre vertical : la feature la plus impactante globalement en haut

Feature Importance	SHAP
Vision globale sur le dataset entier	Vision locale par prédiction individuelle
"Quelle feature est la plus utilisée ?"	"Pourquoi cette machine précise est-elle à risque ?"
Basée sur la fréquence d'utilisation	Basée sur la contribution marginale

Utilité opérationnelle : face à un responsable maintenance qui demande pourquoi telle machine est classée à haut risque, SHAP permet de répondre concrètement : "La vibration RMS est anormalement élevée depuis ce matin et la dernière maintenance remonte à 30 jours. Ce sont les deux facteurs qui déclenchent l'alerte."

11.3 Analyse métier des décisions du modèle

Le modèle XGBoost a appris des règles de décision qui correspondent à la physique des machines industrielles. Une vibration RMS élevée combinée à une température moteur élevée constitue le signal le plus fort de panne imminente.

Un `hours_since_maintenance` élevé augmente le risque de façon continue, reflétant l'usure cumulative des pièces entre deux interventions. Le mode opératoire `peak` est associé à un risque plus élevé, ce qui est attendu compte tenu du stress mécanique supérieur qu'il génère.

Ces correspondances entre les décisions du modèle et la physique des équipements permettent à un responsable maintenance de faire confiance aux alertes et de comprendre leur origine sans avoir besoin de comprendre les algorithmes.

12. Interface utilisateur et dashboard décisionnel

Le dashboard Streamlit a été conçu comme un outil décisionnel, distinct des visualisations d'analyse scientifique de l'EDA. Il cible un responsable maintenance non technique qui a besoin d'une réponse, pas d'un graphique de corrélation.

12.1 Architecture du dashboard

Onglet	Contenu	Valeur métier
EDA	Distributions, corrélations, valeurs manquantes	Compréhension du comportement des machines
Modèles	Tableau comparatif, courbes ROC, matrices de confusion	Confiance dans la fiabilité du système
Prédiction	Formulaire capteurs, score de risque	Usage opérationnel quotidien
Interprétabilité	Feature importance, SHAP	Explication de chaque alerte

12.2 Prédiction en temps réel

L'utilisateur saisit les valeurs des capteurs d'une machine suspecte. Le dashboard affiche immédiatement :

- Un score de risque coloré (ÉLEVÉ / MODÉRÉ / FAIBLE)
- La probabilité exacte de panne en pourcentage
- Une recommandation en langage naturel (par exemple : "Intervention recommandée dans les 24h")

12.3 Différence entre visualisations EDA et visualisations métier

Les graphiques de l'EDA (section 5) sont des visualisations d'analyse scientifique : histogrammes, heatmaps, boxplots. Ils ont été produits pour le data scientist qui cherche à comprendre les données avant de modéliser.

Les visualisations du dashboard sont orientées vers la décision : score coloré, jauge de probabilité, liste des facteurs de risque en langage naturel.

12.4 Lancement

```
streamlit run dashboard/app.py
```

13. Résultats et tests de démonstration

Deux scénarios d'usage ont été testés pour valider le comportement du système sur des cas contrastés.

13.1 Scénario 1 — Machine à faible risque

Paramètre	Valeur saisie	Contexte
vibration_rms	1.2	Valeur normale
temperature_motor	68.0	Température dans la plage normale
pressure_level	4.5	Pression stable
rpm	1450	Régime nominal
hours_since_maintenance	12.0	Maintenance récente
machine_type	CNC	Type standard
operating_mode	normal	Mode nominal

Résultat obtenu : probabilité de panne entre 5 et 15 %, score FAIBLE (vert), recommandation de surveillance normale.

Les capteurs sont dans leurs plages habituelles et la machine a été entretenue récemment. Le modèle prédit correctement l'absence de risque immédiat.

13.2 Scénario 2 — Machine à haut risque

Paramètre	Valeur saisie	Contexte
vibration_rms	4.8	Vibration anormalement élevée
temperature_motor	112.0	Surchauffe moteur
pressure_level	7.2	Pression élevée
rpm	2800	Régime en surcharge
hours_since_maintenance	720.0	Maintenance ancienne (30 jours)
machine_type	Pump	Type à haute sollicitation
operating_mode	peak	Mode surcharge

Résultat obtenu : probabilité de panne entre 85 et 95 %, score ÉLEVÉ (rouge), recommandation d'intervention dans les 24h.

La combinaison vibration élevée, surchauffe, maintenance ancienne et mode peak active simultanément les trois principaux signaux prédictifs. L'analyse SHAP confirme que `vibration_rms` et `temperature_motor` sont les contributions dominantes à cette prédiction.

13.3 Cohérence et comportement aux cas limites

Les prédictions sont cohérentes avec ce qu'un technicien de maintenance attendrait : les variables que SHAP identifie comme déterminantes correspondent aux indicateurs physiques classiques de dégradation des machines industrielles en général.

Avec une Precision de 0.847, environ une alerte sur six est une fausse alarme. Dans un contexte industriel, ce taux est acceptable si le coût d'une intervention préventive inutile reste inférieur au coût d'une panne non anticipée, ce qui est sûrement le cas.

Pour les cas ambigus (valeurs modérées sur toutes les variables), le modèle produit une probabilité entre 40 et 60 %, ce qui permet à l'utilisateur de prendre une décision en prenant en compte l'incertitude plutôt que de recevoir une réponse binaire pouvant être trompeuse.

14. Gouvernance, responsabilité et limites

14.1 Qualité des données et traçabilité

L'intégralité du preprocessing est encapsulée dans des sklearn Pipelines sérialisés avec joblib. Chaque étape (imputation, normalisation, encodage) est reproductible et peut être auditée. Le code source est versionné sur GitHub, ce qui permet de recréer exactement les mêmes résultats à partir des données brutes.

14.2 Risques de biais et limites du modèle

Les quatre types de machines sont représentés équitablement dans le dataset. Toutefois, des équipements d'une nouvelle ligne de production non représentée pendant l'entraînement pourraient produire des prédictions moins fiables.

En production réelle, les capteurs vieillissent et leurs distributions changent progressivement. Un modèle entraîné en 2024 pourrait voir ses performances se dégrader silencieusement sans que personne ne s'en aperçoive, faute de monitoring adapté.

Avec un Recall de 0.955, environ 4.5 % des pannes réelles ne seront pas détectées.

14.3 Enjeux de responsabilité

Le système est un outil d'aide à la décision, pas un système autonome. Les alertes générées par le modèle doivent être validées par un ingénieur maintenance avant toute décision critique (arrêt de production, remplacement de pièces). Notre modèle n'est pas infallible.

14.4 Recommandations pour un déploiement responsable

Action	Priorité	Objectif
Monitoring de dérive des capteurs (data drift)	Haute	Détecter une dégradation des performances en production
Réentraînement périodique avec nouvelles données	Haute	Maintenir la pertinence du modèle dans le temps
Validation humaine pour les décisions critiques	Haute	Garder l'expert dans la boucle de décision
Documentation technique du pipeline	Moyenne	Auditabilité et reproductibilité
Ajustement du seuil de décision selon les coûts réels	Haute	Adapter le compromis Recall / Precision au contexte opérationnel

15. Limites et pistes d'amélioration

15.1 Limites du dataset

Le dataset est synthétique, ce qui simplifie le problème par rapport à des données industrielles réelles où le bruit de capteur, les pannes atypiques et les comportements spécifiques à chaque équipement compliquent la modélisation.

Chaque enregistrement est traité comme un instantané indépendant. Les tendances de dégradation progressive (une vibration qui monte lentement sur six heures) ne sont pas capturées, alors qu'elles constituent souvent le signal le plus précoce d'une panne à venir.

Des informations contextuelles potentiellement utiles sont absentes du dataset : l'âge des machines, leur historique complet de pannes et la qualité des pièces de remplacement utilisées.

15.2 Limites méthodologiques

Le seuil de décision est fixé à 0.5 selon notre arbitrage. En production, ce seuil devrait être calibré en fonction du ratio coût d'une panne non détectée / coût d'une intervention préventive inutile. Si une panne coûte 10 fois plus qu'une fausse alerte, abaisser le seuil à 0.3 permettrait d'augmenter significativement le Recall.

Aucun monitoring de dérive des données n'est en place. En production, les distributions des capteurs évoluent avec le vieillissement des machines, et les performances du modèle se dégraderaient progressivement sans que rien ne le signale.

Les hyperparamètres ont été fixés manuellement. Une recherche par RandomizedSearchCV pourrait améliorer les performances, même marginalement.

15.3 Comparaison ML vs Deep Learning

Facteur	Explication
Taille du dataset (24 000 obs.)	Les réseaux de neurones ont besoin de 100 000 observations minimum pour exploiter leur capacité
Features tabulaires structurées	Les arbres de décision sont naturellement adaptés à ce format
Features déjà bien définies par des capteurs	Peu de valeur ajoutée dans l'apprentissage de représentations intermédiaires
Données déséquilibrées	XGBoost gère le déséquilibre via <code>scale_pos_weight</code> , plus directement qu'un réseau

XGBoost (F1 = 0.898) surpasse le MLP (F1 = 0.850).

15.4 Pistes d'amélioration prioritaires

Amélioration	Impact estimé	Complexité
Rolling features (moyenne sur 1h, 6h, 24h)	+3 à 5 % de F1	Moyenne
Ajustement du seuil de décision	+5 à 10 % de Recall	Faible
Monitoring data drift (Evidently AI)	Fiabilité long terme	Moyenne
LSTM sur séries temporelles	Potentiellement +5 à 10 %	Élevée
Déploiement cloud (AWS ou GCP)	Accessibilité en production	Élevée

16. Conclusion et recommandations

16.1 Bilan

Le projet a couvert l'intégralité de la chaîne Data Science, de la compréhension du besoin métier jusqu'à un outil opérationnel :

- Analyse du besoin : utilisateur cible identifié, scénarios d'usage définis
- EDA : patterns physiques validés dans les données, déséquilibre géré
- Preprocessing : pipeline sklearn anti-leakage, reproductible et traçable
- Modélisation : quatre algorithmes comparés (LR, RF, XGBoost, MLP)
- Évaluation : XGBoost retenu avec F1 = **0.898**, Recall = **0.955**, ROC-AUC = **0.996**
- Interprétabilité : SHAP valide la cohérence physique des décisions du modèle
- Dashboard : Streamlit opérationnel, 4 onglets, prédiction en temps réel

16.2 Recommandations pour une mise en production

Recommandation	Priorité	Impact attendu
Ajuster le seuil de décision selon le ratio coût panne / coût intervention	Haute	+5 à 10 % de Recall
Intégrer des features temporelles (rolling mean 1h, 6h, 24h)	Haute	+3 à 5 % de F1 estimé
Mettre en place un monitoring de dérive des distributions	Moyenne	Fiabilité long terme
Prévoir un réentraînement périodique avec les nouvelles données de production	Moyenne	Maintien des performances
API REST FastAPI pour intégration SCADA/ERP	Optionnel	Industrialisation

16.3 Impact métier estimé

Un Recall de 95.5 % signifie que le système détecte 19 pannes sur 20 avant qu'elles surviennent. Dans un environnement industriel où chaque panne non prévue entraîne un arrêt de ligne, des réparations d'urgence et des risques opérateurs, cette capacité de détection précoce a une valeur économique directe.

16.4 Ce que ce projet nous a appris

Au-delà des performances, ce projet illustre que les choix méthodologiques comptent autant que le choix de l'algorithme. La suppression des variables à risque de leakage, la gestion du déséquilibre de classes, le paramétrage du seuil de décision très important du point de vue métier et l'explicabilité des résultats via SHAP sont des décisions qui déterminent si une solution Data Science peut réellement être utilisée en production ou si elle reste un exercice académique.